TUEVENTO · CAPYSOFT

# Registros de decisiones de arquitectura (ADR, repositorio yev-docs)

Plataforma web y móvil para la gestión de eventos y la venta de boletas

Análisis y Desarrollo de Software (228118) · Ficha 3145556
Proyecto formativo: Desarrollo de software orientado a servicios
SENA · Centro de la Industria, la Empresa y los Servicios · Regional Huila
Aprendices: Cristofer David Lozano Contreras · Jhampier Santos Ortiz
Versión entregada: final-delivery-2026 · Noviembre de 2026

# ADR-001 — Documentation Language

| Field | Value |
|---|---|
| **ID** | ADR-001 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento is a multilingual event management platform supporting Spanish (ES), English (EN), Portuguese (PT), and Japanese (JA) through a comprehensive internationalization system. However, the technical documentation, codebase, and development artifacts require a consistent language strategy.

The software industry standard — Stack Overflow, library documentation, technical articles, frameworks, and open-source tooling — operates in English. Mixing languages across artifacts (Spanish docs, English code, Spanish API descriptions) creates a cognitive translation boundary that slows onboarding, increases errors in naming, and makes it harder to search for help online.

A single, clear rule established from day one prevents inconsistencies: mismatched variable names, Spanish comments in English code, and conflicting terminology between documentation and implementation.

**TuEvento's multilingual architecture:**

- **End-user content**: Translated via `language` table and `*_translation` tables
- **Technical artifacts**: Need consistent language for development team
- **Database schema**: 14+ translation tables supporting dynamic content localization
- **Business domain**: Colombian market with Spanish business terms

---

## Evaluated alternatives

### Alternative A — Everything in English (CHOSEN)

- **Pros:**
  - Industry standard for technical documentation and code
  - Easy to hire external developers and integrate with international teams
  - Libraries and frameworks documentation available in English
  - Eliminates translation boundary between technical docs and code
  - GitHub, Stack Overflow, and AI tools work optimally with English content

    - Spring Boot, PostgreSQL, React ecosystem all documented in English
    - Error messages and logs searchable in English documentation
- **Cons:**
    - May require more effort from team members less confident in English
    - Business domain terms need careful translation from Spanish originals

## Alternative B — Everything in Spanish

- **Pros:**
    - Natural communication with Colombian business stakeholders
    - Domain terms like "silla", "evento", "boleto" preserved exactly as understood by users
    - Easier for team members whose primary language is Spanish
- **Cons:**
    - Creates uncomfortable mix with English keywords (if, for, return, class, etc.)
    - Inconsistent with library ecosystem (Spring Boot, React, PostgreSQL docs)
    - External contributors cannot participate effectively
    - Technical problem-solving requires translating error messages and concepts
    - Database naming becomes inconsistent with ORM conventions

## Alternative C — Split by Audience (discarded)

**Technical artifacts in English, business documentation in Spanish**

- **Pros:**
    - Each artifact uses most natural language for its audience
    - Business stakeholders get Spanish documentation
    - Developers get English technical docs
- **Cons:**
    - Requires strict discipline and explicit rules for every artifact type
    - Creates permanent translation boundary between business and technical domains
    - Domain terms accumulate two canonical names (Spanish business + English technical)
    - Onboarding complexity increases significantly

## Alternative D — Hybrid with Glossary (discarded)

**English for technical, Spanish for domain, with comprehensive glossary**

- **Pros:**
    - Preserves business domain authenticity
    - Maintains technical documentation standards
- **Cons:**
    - High maintenance overhead for glossary
    - Still creates cognitive translation boundary
    - Inconsistency leads to errors and confusion

## Decision

**Alternative A:** Use English for all technical documentation, code, and development artifacts.

| Artifact Category | Language | Justification |
|---|---|---|
| **Code (variables, functions, classes)** | English | Consistency with Java/Spring conventions and international libraries |
| **Database schema (tables, columns)** | English | Coherence with JPA entity mappings and Spring Data repositories |
| **API contracts (OpenAPI descriptions)** | English | Industry standard for REST APIs, readable by international consumers |
| **Git commits (Conventional Commits)** | English | Established standard, readable on GitHub, searchable |
| **Branch names and PR titles** | English | Consistent with commits and GitHub ecosystem |
| **Technical documentation (ADRs, architecture)** | English | Eliminates translation boundary, searchable, reusable by other teams |
| **Error messages and logs** | English | Facilitates search in library documentation and Stack Overflow |
| **Configuration files and deployment scripts** | English | Standard practice in DevOps and infrastructure tools |

**End-user facing content remains multilingual:**

- **Event names, descriptions**: Translated via `event_translation` table
- **UI labels and messages**: Handled by frontend internationalization
- **Notifications**: Stored in multiple languages via `notification_translation`
- **Categories and metadata**: Support all four languages (ES/EN/PT/JA)

**Domain term mapping:** Critical business terms maintain English canonical form in technical artifacts:

| Spanish Business Term | English Technical Term | Usage Context |
|---|---|---|
| Evento | Event | Database tables, API endpoints, class names |
| Silla/Asiento | Seat | Entity names, reservation logic |
| Boleto/Entrada | Ticket | Payment processing, QR generation |
| Organizador | Organizer | Role-based access control |
| Sección | Section | Venue layout, seating management |

## Consequences

**Positive:**

- **Single source of truth**: Domain terms have one canonical form in the codebase (English)
- **Reduced cognitive load**: No mental translation required between docs and implementation

- **External collaboration**: International developers can contribute without language barriers
- **Problem-solving efficiency**: Error messages directly searchable in English documentation
- **AI tool compatibility**: ChatGPT, GitHub Copilot work optimally with English code comments
- **Recruitment flexibility**: Easier to onboard developers regardless of Spanish proficiency

**Negative:**

- **Learning curve**: Team members need to map Spanish business concepts to English technical terms
- **Business communication**: Requires translation layer when discussing technical implementation with Spanish-speaking stakeholders

**Mitigation strategies:**

- **Comprehensive glossary**: Maintain `01-context/glossary.md` with Spanish ↔ English business term mappings
- **Documentation standards**: All technical documentation includes business context in both languages where relevant
- **Code comments**: Use English for technical comments, include Spanish business context when helpful
- **Database documentation**: Include business meaning for technical entities

**Implementation guidelines:**

```
// GOOD: English technical terms with business context
@Entity
@Table(name = "event")
public class Event {
    @Column(name = "event_name")
    private String name; // "Nombre del evento" in Spanish UI

    @Column(name = "start_date")
    private LocalDateTime startDate; // "Fecha de inicio"
}

// GOOD: English API endpoints
@GetMapping("/events/{eventId}/seats")
public ResponseEntity<List<SeatDto>> getEventSeats(@PathVariable String eventId) {
    // Implementation
}

// GOOD: English commit messages
// feat(seating): add real-time seat reservation with WebSocket
// fix(payment): resolve Wompi webhook signature validation
```

**Future considerations:**

- **Team growth**: English documentation enables international team expansion
- **Open source potential**: Technical artifacts ready for open source contribution
- **Market expansion**: English technical foundation supports international market entry

---

## References

- Domain glossary → `01-context/glossary.md`
- Internationalization implementation → `TuEvento/DataBase/MR/DBML/modulo_idiomas.dbml`
- Language module documentation → `modules/language/README.md`
- Team documentation conventions → `00-governance/evidence/documentation-rules.md`
- Frontend internationalization → Frontend localization configuration

# ADR-002 — N-Layer DDD Architecture

| Field | Value |
|---|---|
| **ID** | ADR-002 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento is an event management platform with complex business logic around seat reservations, real-time updates, payment processing, and multi-user coordination. The system needs to handle:

- **Complex domain logic**: Event creation, seat layouts, reservations, tickets, payments
- **Real-time requirements**: Seat status updates via WebSockets
- **Multiple bounded contexts**: Security, Events, Payments, Notifications each with distinct rules
- **External integrations**: Wompi payments, SMTP notifications, OAuth providers
- **Multi-platform access**: Web organizers, mobile/web attendees
- **Small team size**: 2 developers need maintainable, well-structured code

We need an architecture that balances domain complexity with development velocity and maintainability.

---

## Evaluated alternatives

### Alternative A — N-Layer with Domain-Driven Design (CHOSEN)

**Structure:** Each module follows domain/application/infrastructure/interfaces layers within a Spring Boot monolith

- **Pros:**
  - Clear separation of concerns with domain isolation
  - Bounded contexts map naturally to modules
  - Testable business logic independent of frameworks
  - Supports future microservices evolution
  - Familiar to team, good Spring Boot ecosystem support
  - Single deployment simplifies DevOps for small team
- **Cons:**
  - More complex initial setup than simple layered architecture

    - Requires discipline to maintain layer boundaries
    - Potential for over-engineering simple CRUD operations

### Alternative B — Simple 3-Layer Architecture (MVC)

**Structure:** Controller → Service → Repository pattern across the entire application

- **Pros:**
    - Simplest to implement and understand
    - Fast initial development
    - Standard Spring Boot approach
- **Cons:**
    - Business logic scattered across service classes
    - Difficult to maintain clear bounded context boundaries
    - Hard to test complex domain interactions
    - Doesn't scale well with domain complexity

### Alternative C — Microservices from Day 1

**Structure:** Separate Spring Boot applications per bounded context

- **Pros:**
    - Maximum service isolation
    - Independent deployments
    - Technology diversity possible
- **Cons:**
    - Significant overhead for 2-person team
    - Complex distributed system debugging
    - Network latency for operations spanning contexts
    - Premature optimization for current team size

### Alternative D — Event Sourcing + CQRS

**Structure:** Event-driven architecture with separate read/write models

- **Pros:**
    - Perfect audit trail for financial transactions
    - Natural fit for seat reservation events
    - Excellent for complex business workflows
- **Cons:**
    - High complexity for team experience level
    - Steep learning curve
    - Overkill for current requirements
    - Added infrastructure complexity

## Decision

**Alternative A:** N-Layer Architecture with Domain-Driven Design

Each module follows hexagonal architecture with these layers:

```
modules/[module]/
├── domain/
│   ├── model/           # Entities, Value Objects, Domain Events
│   ├── repository/      # Repository interfaces
│   └── service/         # Domain Services
├── application/
│   ├── usecase/         # Use Cases (Application Services)
│   ├── dto/             # Request/Response DTOs
│   ├── port/            # Input/Output ports
│   └── mapper/          # DTO ↔ Domain mappers
├── infrastructure/
│   ├── persistence/     # JPA entities, Repository implementations
│   ├── messaging/       # Event publishing/handling
│   └── external/        # External service clients
└── interfaces/
    ├── rest/           # REST Controllers
    └── websocket/      # WebSocket endpoints (where needed)
```

**Module boundaries based on bounded contexts:**

- Security (Identity & Access)
- Event (Event Management)
- Seat (Seat Management + Real-time)
- Payment (Payment Processing)
- Ticket (Ticketing System)
- Notification (Communication)
- Profile (User Management)

---

## Consequences

**Positive:**

- **Clear domain boundaries** enable focused development and testing
- **Business logic isolation** makes complex rules maintainable and testable
- **Future microservices path** — modules can be extracted when team grows
- **Technology flexibility** — each module can adopt different patterns as needed
- **Parallel development** — developers can work on different modules with minimal conflicts

- **Testability** — domain logic can be unit tested without Spring context

**Negative:**

- **Initial setup overhead** — more directories and interfaces than simple MVC
- **Learning curve** — team needs to understand DDD concepts and layer responsibilities
- **Potential over-engineering** — simple CRUD modules still need full layer structure

**Mitigation strategies:**

- Create module templates and generator scripts to reduce boilerplate
- Document clear guidelines for what goes in each layer
- Start with simpler modules (Profile, Category) to build team familiarity
- Use shared infrastructure components to avoid duplication

**Migration approach:**

- **Phase 1:** Implement core modules (Security, Event, Seat) with full N-Layer structure
- **Phase 2:** Add remaining modules following established patterns
- **Phase 3:** Extract to microservices when team > 5 developers or scaling requirements demand it

---

## References

- Hexagonal Architecture implementation → `05-architecture/hexagonal-architecture.md`
- Domain model and bounded contexts → `02-domain/evidence/domain-map.md`
- Module catalog and responsibilities → `05-architecture/evidence/overview.md`
- Future microservices strategy → `05-architecture/evidence/overview.md#planned-evolution`

# ADR-003 — Java 23 + Spring Boot Technology Stack

| Field | Value |
|---|---|
| **ID** | ADR-003 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento requires a robust backend technology stack that can handle:

- **Real-time features**: WebSocket connections for seat updates
- **External integrations**: Wompi payment gateway, SMTP, OAuth providers
- **Database operations**: Complex queries, transactions, audit logging
- **Multi-platform APIs**: REST APIs for web and mobile clients
- **Security requirements**: JWT authentication, role-based authorization
- **Performance needs**: Handle concurrent seat reservations, payment processing
- **Team constraints**: 2 developers with Java/Spring experience
- **Development velocity**: Rapid prototyping and iterative development

The technology stack must balance development speed, maintainability, performance, and team expertise.

---

## Evaluated alternatives

### Alternative A — Java 23 + Spring Boot 3.x (CHOSEN)

- **Pros:**
  - Team has strong Java expertise and Spring Boot experience
  - Mature ecosystem with extensive libraries (Security, Data, WebSocket, etc.)
  - Excellent IDE support and debugging tools
  - Built-in support for all required features (REST, WebSocket, JPA, Security)
  - Strong community and documentation
  - Production-ready with battle-tested patterns
  - Spring Boot Actuator for monitoring and health checks
  - Easy integration with PostgreSQL and Redis
  - Maven ecosystem familiar to team
- **Cons:**

    - Verbose compared to newer languages
    - JVM startup time (mitigated by GraalVM in future)
    - Memory footprint higher than compiled languages

### Alternative B — Node.js + Express/NestJS

- **Pros:**
    - JavaScript everywhere (frontend + backend)
    - Fast development for APIs and real-time features
    - Excellent WebSocket support
    - Large npm ecosystem
    - Good performance for I/O-intensive operations
- **Cons:**
    - Team lacks Node.js experience
    - Single-threaded nature challenging for CPU-intensive operations
    - Weaker typing support (even with TypeScript)
    - Less mature ecosystem for enterprise features (security, transactions)
    - Payment integration libraries less robust

### Alternative C — Python + Django/FastAPI

- **Pros:**
    - Clean, readable code
    - Rapid development capabilities
    - Good libraries for data processing and integrations
    - Strong ORM support
- **Cons:**
    - Team has limited Python experience
    - Performance concerns for real-time features
    - GIL limitations for concurrent operations
    - Weaker ecosystem for enterprise payment processing

### Alternative D — C# + .NET Core

- **Pros:**
    - Excellent performance and tooling
    - Strong typing system
    - Good WebSocket and real-time support
    - Mature ecosystem for enterprise applications
- **Cons:**
    - Team has no C# experience
    - Less familiar deployment ecosystem
    - Windows-centric community (though .NET Core is cross-platform)

### Alternative E — Go + Gin/Echo

- **Pros:**
  - Excellent performance and concurrency
  - Fast startup times
  - Good for microservices
  - Strong standard library
- **Cons:**
  - Team has no Go experience
  - Less mature ecosystem for complex business applications
  - Limited ORM options
  - Steeper learning curve for domain-driven design

---

## Decision

**Alternative A:** Java 23 + Spring Boot 3.x

**Core framework stack:**

- **Java 23** — Latest LTS with modern language features
- **Spring Boot 3.2+** — Application framework and auto-configuration
- **Spring Security** — Authentication, authorization, JWT handling
- **Spring Data JPA** — Database access and ORM
- **Spring WebSocket** — Real-time communication with SockJS fallback
- **Spring Web** — REST API development
- **Maven 3.9+** — Dependency management and build tool

**Key libraries:**

- **PostgreSQL Driver** — Primary database connectivity
- **Redis Client (Lettuce)** — Caching and session storage
- **Jackson** — JSON serialization/deserialization
- **Bean Validation (Hibernate Validator)** — Input validation
- **SpringDoc OpenAPI** — API documentation generation
- **Resilience4j** — Circuit breaker for external services
- **Logback** — Structured logging with JSON output

**Development tools:**

- **Spring Boot DevTools** — Hot reloading in development
- **Spring Boot Actuator** — Health checks and monitoring
- **JUnit 5** — Unit and integration testing
- **Testcontainers** — Database integration testing
- **Mockito** — Mocking framework for tests

## Consequences

**Positive:**

- **Team velocity** — Leverages existing Java/Spring expertise for faster development
- **Feature completeness** — Spring ecosystem covers all requirements out-of-the-box
- **Production readiness** — Battle-tested framework with proven scalability
- **Maintainability** — Strong typing, excellent IDE support, comprehensive tooling
- **Integration capabilities** — Mature libraries for all external services (Wompi, SMTP, OAuth)
- **Security** — Spring Security provides enterprise-grade authentication/authorization
- **Real-time support** — Spring WebSocket with SockJS handles cross-browser compatibility
- **Monitoring** — Built-in health checks, metrics, and observability features

**Negative:**

- **Resource usage** — Higher memory footprint than lightweight alternatives
- **Startup time** — Slower application startup compared to compiled languages
- **Verbosity** — More boilerplate code compared to dynamic languages

**Risk mitigation:**

- **Memory optimization** — Use Spring Boot's layered JARs and optimize JVM settings
- **Development speed** — Use Spring Boot starters and auto-configuration to minimize setup
- **Code generation** — Leverage IDE templates and Spring Boot CLI for common patterns
- **Future optimization** — Consider GraalVM native compilation for production if startup time becomes critical

**Technology decisions:**

- **Java 23** chosen over Java 17 LTS for latest language improvements (pattern matching, virtual threads preparation)
- **Spring Boot 3.x** for modern Spring features and GraalVM compatibility
- **Maven** over Gradle for team familiarity and simpler configuration
- **SLF4J + Logback** over other logging frameworks for better JSON structured logging

---

## References

- Backend project structure → `05-architecture/evidence/overview.md#module-catalog`
- Database and caching strategy → `ADR-004-postgresql-redis-strategy.md`
- Security implementation → `ADR-005-jwt-refresh-token-auth.md`
- Real-time communication → `ADR-007-websocket-sockjs.md`
- Development environment setup → `TuEvento/README.md`

# ADR-004 — PostgreSQL + Redis Data Strategy

| Field | Value |
|---|---|
| **ID** | ADR-004 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento requires a data storage strategy that supports:

- **Transactional integrity**: Payment processing, seat reservations must be ACID-compliant
- **Complex relationships**: Events, venues, seats, tickets, users with referential integrity
- **JSON flexibility**: Event layouts, user preferences, notification templates
- **Real-time performance**: Seat status updates, WebSocket session management
- **Audit trails**: Complete history of seat changes, payment transactions
- **Concurrent access**: Multiple users reserving seats simultaneously
- **Search capabilities**: Event filtering, user management
- **Caching requirements**: Session storage, frequently accessed data
- **Backup and recovery**: Financial data protection and compliance

The team has PostgreSQL experience and needs a solution that balances ACID compliance with performance.

---

## Evaluated alternatives

### Alternative A — PostgreSQL + Redis (CHOSEN)

**PostgreSQL as primary database, Redis for caching and sessions**

- **Pros:**
  - ACID compliance for financial transactions and reservations
  - Excellent JSON/JSONB support for flexible schemas (event layouts)
  - Strong referential integrity and foreign key constraints
  - Advanced indexing (B-tree, GIN, GiST) for complex queries
  - Team expertise and familiarity
  - Robust backup and point-in-time recovery

    - Excellent Spring Data JPA integration
    - PostgreSQL 17 performance improvements
    - Redis provides high-performance caching and session storage
    - Redis supports WebSocket session management and pub/sub
- **Cons:**
    - Two systems to maintain and monitor
    - Network calls for cache operations
    - Redis adds complexity for small team

## Alternative B — PostgreSQL Only

**Single database approach with PostgreSQL extensions**

- **Pros:**
    - Single system to maintain and backup
    - No network latency for cached data
    - PostgreSQL can handle moderate caching via connection pooling
    - Simpler deployment and monitoring
- **Cons:**
    - WebSocket session state would need database storage (less efficient)
    - No distributed caching capabilities
    - Higher load on primary database for frequent reads
    - Session storage in database affects performance

## Alternative C — MySQL + Redis

**MySQL as primary database with Redis caching**

- **Pros:**
    - Wide adoption and extensive documentation
    - Good JSON support in recent versions
    - Redis caching benefits
- **Cons:**
    - Team lacks MySQL expertise
    - Weaker JSON handling compared to PostgreSQL
    - Less advanced indexing capabilities
    - Migration effort from team's PostgreSQL knowledge

## Alternative D — MongoDB + Redis

**Document database with caching layer**

- **Pros:**
    - Natural fit for JSON-heavy data (event layouts)

    - Flexible schema evolution
    - Good performance for read-heavy workloads
- **Cons:**
    - Team has no MongoDB experience
    - Weaker ACID guarantees for multi-document transactions
    - Less suitable for complex relational data (users, payments)
    - Limited SQL tooling and reporting capabilities

### Alternative E — Microservice Databases

**Separate databases per bounded context**

- **Pros:**
    - Perfect service isolation
    - Technology diversity per service needs
    - Independent scaling
- **Cons:**
    - Complex distributed transactions
    - Data consistency challenges
    - Multiple database systems to maintain
    - Premature for 2-person team

---

## Decision

**Alternative A:** PostgreSQL 17 as primary database + Redis 7.x for caching and sessions

**PostgreSQL configuration:**

- **Primary database**: `tuevento_db` with all business data
- **Connection pooling**: HikariCP with optimized settings
- **JSON storage**: JSONB for event layouts, user preferences, notification templates
- **Indexing strategy**: GIN indexes for JSONB queries, B-tree for standard lookups
- **Audit logging**: Trigger-based audit tables for critical operations
- **Backup strategy**: Daily automated backups with point-in-time recovery

**Redis configuration:**

- **Session storage**: Spring Session with Redis for JWT refresh tokens
- **WebSocket sessions**: Redis pub/sub for real-time seat updates
- **Application cache**: Frequently accessed data (event details, user profiles)
- **Temporary data**: Seat reservations with TTL (15-minute expiration)

**Schema design principles:**

- **Single database**: All modules share `tuevento_db` with clear schema separation
- **Foreign key constraints**: Enforce referential integrity between modules
- **Audit tables**: Separate audit schema for compliance and debugging
- **JSON fields**: Use JSONB for semi-structured data (layouts, preferences)

---

## Consequences

**Positive:**

- **ACID compliance** ensures data integrity for payments and reservations
- **Team expertise** leverages existing PostgreSQL knowledge for faster development
- **JSON flexibility** supports complex event layouts and user preferences
- **Performance** Redis caching reduces database load and improves response times
- **Real-time support** Redis pub/sub enables efficient WebSocket scaling
- **Backup/Recovery** PostgreSQL provides enterprise-grade data protection
- **Query capabilities** Advanced PostgreSQL features support complex business queries
- **Spring integration** Excellent Spring Boot support for both technologies

**Negative:**

- **Complexity** Two systems require separate monitoring and maintenance
- **Network latency** Cache operations add network round-trips
- **Memory usage** Redis requires dedicated memory allocation
- **Consistency** Cache invalidation needs careful coordination

**Performance optimizations:**

- **Connection pooling** HikariCP with proper sizing for concurrent access
- **Query optimization** Use EXPLAIN ANALYZE for complex queries, add appropriate indexes
- **Cache strategy** Write-through caching for frequently read data
- **Redis clustering** Prepare for Redis Sentinel/Cluster when scaling requirements grow

**Migration and deployment:**

- **Development** Docker Compose with PostgreSQL and Redis containers
- **Production** Managed PostgreSQL (cloud) + Redis instance with backup strategies
- **Monitoring** PostgreSQL slow query log, Redis monitoring via Spring Actuator

**Schema evolution:**

- **Database migrations** Flyway for versioned schema changes
- **Backward compatibility** Careful planning for schema changes affecting JSON fields
- **Module separation** Clear table prefixes and schemas for each bounded context

---

## References

- Database schema design → `06-data/models.md`
- Real-time seat management → `ADR-007-websocket-sockjs.md`
- JWT session management → `ADR-005-jwt-refresh-token-auth.md`
- Deployment configuration → `TuEvento/README.md#getting-started`
- Module data models → `02-domain/evidence/entities-and-rules.md`

# ADR-005 — JWT + Refresh Token Authentication

| Field | Value |
|---|---|
| **ID** | ADR-005 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento requires a secure authentication system that supports:

- **Multi-platform access**: Web (Vite), Mobile (Expo), future API integrations
- **Stateless scalability**: No server-side session storage dependency
- **Security requirements**: Secure token handling, account lockout, password recovery
- **User roles**: USER, ORGANIZER, ADMIN with different permissions
- **Social login**: OAuth integration with Google, Facebook
- **Mobile-friendly**: Long-lived sessions without frequent re-authentication
- **Security compliance**: Protection against token theft, replay attacks
- **Performance**: Fast authentication checks without database hits

The solution must balance security, user experience, and development simplicity for a small team.

---

## Evaluated alternatives

### Alternative A — JWT + Refresh Token (CHOSEN)

#### Short-lived JWT access tokens + long-lived refresh tokens stored in Redis

- **Pros:**
  - Stateless authentication - no server session storage required
  - Mobile-friendly with automatic token refresh
  - Scalable - tokens contain all necessary user information
  - Cross-platform compatibility (web, mobile, API)
  - Industry standard with extensive Spring Security support
  - Social login integration straightforward
  - Short access token lifetime limits damage from token theft
  - Refresh tokens can be revoked centrally

- **Cons:**
  - More complex implementation than sessions
  - Refresh token management adds Redis dependency
  - Token size larger than session cookies

## Alternative B — Traditional Sessions + Cookies

**Server-side sessions stored in Redis with HttpOnly cookies**

- **Pros:**
  - Simpler implementation with Spring Session
  - Smaller request overhead (session ID only)
  - Easy session invalidation
  - Built-in CSRF protection
- **Cons:**
  - Mobile app complications with cookie handling
  - CORS complexity for cross-domain requests
  - Less suitable for API integrations
  - Session storage scaling challenges
  - Mobile platforms don't handle cookies naturally

## Alternative C — OAuth-Only (No local auth)

**Delegate all authentication to external OAuth providers**

- **Pros:**
  - No password management complexity
  - Leverages user's existing accounts
  - Reduced security liability
- **Cons:**
  - External dependency for core functionality
  - User experience issues when OAuth providers unavailable
  - Limited control over user management
  - Not all users have social accounts
  - Business model constraints (platform dependency)

## Alternative D — API Keys

**Simple API key-based authentication**

- **Pros:**
  - Very simple to implement
  - No token expiration complexity
- **Cons:**

- No user identity/role information
- Difficult key rotation
- Not suitable for end-user applications
- Poor security practices for user-facing auth

### Alternative E — Session + JWT Hybrid

**Sessions for web, JWT for mobile**

- **Pros:**
  - Optimized per platform
  - Leverages best of both approaches
- **Cons:**
  - Dual implementation complexity
  - Inconsistent security model
  - More testing scenarios
  - Code duplication and maintenance overhead

---

## Decision

**Alternative A:** JWT Access Tokens + Refresh Token pattern

**Authentication flow:**

```
1. Login (email/password or OAuth) → Server validates credentials
2. Server generates:
   - JWT access token (1 hour expiration)
   - Refresh token (7 days expiration)
3. Access token sent in response, refresh token stored in Redis
4. Client uses access token for API requests (Authorization: Bearer)
5. When access token expires, client uses refresh token to get new pair
6. Refresh tokens can be revoked for logout/security
```

**JWT access token payload:**

```
{
  "sub": "user-uuid",
  "username": "user@example.com",
  "roles": ["USER", "ORGANIZER"],
  "iat": 1693123456,
  "exp": 1693127056
}
```

**Security measures:**

- **Access tokens**: 1-hour expiration, stateless validation
- **Refresh tokens**: 7-day expiration, stored in Redis with user mapping
- **Token rotation**: New refresh token issued on each refresh
- **Revocation**: Refresh tokens deleted from Redis on logout
- **Account lockout**: 5 failed attempts → 30-minute lockout
- **Password requirements**: Minimum 8 chars, uppercase, lowercase, number, special char

**Implementation details:**

- **JWT library**: Spring Security OAuth2 Resource Server
- **Token storage**: Redis with TTL matching token expiration
- **Signing algorithm**: HS256 with secure random secret
- **Role mapping**: Spring Security authorities from JWT claims
- **Social login**: Spring Security OAuth2 Client for Google/Facebook

---

## Consequences

**Positive:**

- **Scalability** — Stateless tokens eliminate session storage bottlenecks
- **Mobile support** — Natural token handling in mobile applications
- **Security** — Short access token lifetime limits exposure window
- **Flexibility** — Easy integration with future API consumers
- **Performance** — No database lookup for authentication checks
- **Cross-platform** — Consistent authentication across web and mobile
- **Social integration** — Straightforward OAuth provider integration

**Negative:**

- **Complexity** — More implementation complexity than simple sessions
- **Token size** — Larger request headers compared to session cookies
- **Refresh logic** — Client applications must handle token refresh
- **Redis dependency** — Refresh token storage requires Redis availability

**Security considerations:**

- **Token storage**: Mobile apps use secure storage (Keychain/Keystore)
- **Web storage**: Access tokens in memory only, refresh tokens in HttpOnly cookies
- **HTTPS enforcement**: All token transmission over secure connections
- **Token validation**: Signature verification and expiration checks on every request
- **Refresh rotation**: Prevents refresh token reuse attacks

**Implementation phases:**

- **Phase 1**: Basic JWT authentication with email/password

- **Phase 2**: Refresh token mechanism and proper token rotation
- **Phase 3**: OAuth social login integration
- **Phase 4**: Account lockout and security policies

**Monitoring and security:**

- **Failed login tracking**: Monitor for brute force attempts
- **Token usage analytics**: Track token refresh patterns for anomaly detection
- **Security alerts**: Unusual login locations or device patterns

---

## References

- Security module implementation → `modules/security/`
- Account lockout policies → `00-governance/evidence/security-rules.md`
- Redis session configuration → `ADR-004-postgresql-redis-strategy.md`
- API authentication patterns → `07-api/contracts/openapi/_shared.yaml`
- OAuth provider configuration → `modules/security/infrastructure/external/`

# ADR-006 — Wompi Payment Gateway Integration

| Field | Value |
|---|---|
| **ID** | ADR-006 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento requires a secure, reliable payment processing solution for the Colombian market that supports:

- **Multiple payment methods**: Credit/debit cards, PSE (bank transfers), Nequi, cash payments
- **Colombian market compliance**: Local regulations and tax requirements
- **Security standards**: PCI DSS compliance for card processing
- **Real-time processing**: Immediate payment confirmation for seat reservations
- **Mobile optimization**: Smooth payment experience on mobile devices
- **Webhook notifications**: Real-time payment status updates
- **Refund capabilities**: Support for event cancellations and refunds
- **Competitive rates**: Reasonable processing fees for small/medium events

The payment gateway must integrate seamlessly with the seat reservation system to prevent double bookings.

---

## Evaluated alternatives

### Alternative A — Wompi (CHOSEN)

**Colombian payment processor with comprehensive local support**

- **Pros:**
  - **Colombian market leader**: Deep understanding of local payment preferences
  - **Multiple payment methods**: Cards, PSE, Nequi, Daviplata, cash (Efecty, Baloto)
  - **Competitive rates**: Lower fees than international providers for Colombian market
  - **PCI DSS compliant**: Handles sensitive card data processing
  - **Excellent documentation**: Well-documented REST API with SDKs
  - **Mobile-optimized**: Native mobile payment flows
  - **Real-time webhooks**: Immediate payment status notifications

    - **Tokenization**: Secure card tokenization for recurring payments
    - **Colombian peso native**: No currency conversion issues
    - **Local support**: Spanish-speaking support team in Colombian timezone
- **Cons:**
    - **Geographic limitation**: Primarily focused on Colombian market
    - **Smaller ecosystem**: Less third-party tooling compared to global providers
    - **API rate limits**: More restrictive than larger international providers

## Alternative B — PayU Colombia

**International payment provider with Colombian presence**

- **Pros:**
    - **International backing**: Part of larger Prosus/Naspers group
    - **Multiple countries**: Experience across Latin America
    - **Comprehensive features**: Advanced fraud detection, recurring payments
    - **Enterprise features**: Better reporting and analytics tools
- **Cons:**
    - **Higher costs**: More expensive fees, especially for smaller transactions
    - **Complex integration**: More complex API with steeper learning curve
    - **Limited local methods**: Weaker support for local payment methods (Nequi, Daviplata)
    - **International focus**: Less optimization for Colombian market specifics

## Alternative C — Stripe

**Global payment leader with international expansion**

- **Pros:**
    - **Excellent developer experience**: Outstanding documentation and SDKs
    - **Rich ecosystem**: Extensive third-party integrations and tools
    - **Advanced features**: Sophisticated fraud detection, payment optimization
    - **Global reach**: Easy expansion to other markets
- **Cons:**
    - **Limited Colombian support**: No PSE, Nequi, or cash payment support
    - **Higher costs**: International processing fees
    - **Currency conversion**: USD/COP conversion adds complexity and fees
    - **Compliance**: Additional complexity for Colombian tax reporting

## Alternative D — MercadoPago Colombia

**Latin American payment provider owned by MercadoLibre**

- **Pros:**
    - **Regional expertise**: Strong presence in Latin America

    - **Local payment methods**: Good support for Colombian payment preferences
    - **E-commerce integration**: Strong integration with online marketplaces
    - **Brand recognition**: Well-known brand in Colombian market
- **Cons:**
    - **Marketplace focus**: Optimized for marketplace transactions, not events
    - **Limited customization**: Less flexible for custom payment flows
    - **Documentation quality**: Less comprehensive technical documentation
    - **Support quality**: Mixed reviews on technical support responsiveness

### Alternative E — Custom Payment Processing

**Direct integration with Colombian banks and payment networks**

- **Pros:**
    - **Maximum control**: Complete control over payment flow and user experience
    - **Lower long-term costs**: No payment processor fees
    - **Custom features**: Tailored functionality for specific business needs
- **Cons:**
    - **Massive complexity**: Requires PCI DSS certification, bank integrations
    - **Development time**: Months of development vs. weeks with existing processor
    - **Compliance burden**: Direct responsibility for security and regulatory compliance
    - **Maintenance overhead**: Ongoing maintenance of complex payment infrastructure
    - **Risk**: High risk of security vulnerabilities and compliance violations

---

## Decision

**Alternative A:** Wompi Payment Gateway Integration

**Integration architecture:**

```
TuEvento → Wompi API → Colombian Payment Networks
    ↓
Wompi Webhooks → TuEvento (payment status updates)
```

**Supported payment methods:**

- **Credit/Debit Cards**: Visa, Mastercard, American Express, Diners Club
- **PSE (Pagos Seguros en Línea)**: Direct bank transfers via ACH Colombia
- **Digital wallets**: Nequi, Daviplata
- **Cash payments**: Efecty, Baloto, Su Red network
- **International cards**: For international attendees

**Key integration features:**

- **Tokenization**: Secure storage of payment methods for future use
- **3D Secure**: Additional security layer for card payments
- **Real-time webhooks**: Immediate payment status notifications
- **Refund API**: Automated refund processing for event cancellations
- **Fraud protection**: Built-in fraud detection and prevention

**Technical implementation:**

```
@Service
public class WompiPaymentService {

    public PaymentResponse processPayment(PaymentRequest request) {
        // 1. Create payment intent with Wompi
        WompiPaymentIntent intent = wompiApiClient.createPaymentIntent(
            WompiPaymentIntentRequest.builder()
                .amountInCents(request.getAmountInCents())
                .currency("COP")
                .reference(request.getOrderId())
                .customerEmail(request.getCustomerEmail())
                .redirectUrl(request.getRedirectUrl())
                .build()
        );

        // 2. Store payment record
        Payment payment = Payment.builder()
            .orderId(request.getOrderId())
            .wompiPaymentIntentId(intent.getId())
            .status(PaymentStatus.PENDING)
            .amount(request.getAmountInCents())
            .currency("COP")
            .build();
        paymentRepository.save(payment);

        // 3. Return payment URL for user redirect
        return PaymentResponse.builder()
            .paymentUrl(intent.getPaymentUrl())
            .paymentIntentId(intent.getId())
            .build();
    }

    @EventListener
    public void handleWompiWebhook(WompiWebhookEvent event) {
        // Verify webhook signature
        if (!wompiWebhookValidator.isValidSignature(event)) {
            log.warn("Invalid Wompi webhook signature");
            return;
        }
```

```
        // Process payment status update
        Payment payment = paymentRepository
            .findByWompiPaymentIntentId(event.getPaymentIntentId())
            .orElseThrow(() -> new PaymentNotFoundException("Payment not found"));

        payment.updateStatus(event.getStatus());
        payment.setWompiTransactionId(event.getTransactionId());
        paymentRepository.save(payment);

        // Publish domain event for other modules
        eventPublisher.publishEvent(new PaymentStatusUpdatedEvent(
            payment.getId(), payment.getStatus(), payment.getOrderId()
        ));
    }
}
```

---

## Consequences

**Positive:**

- **Local market fit**: Perfect alignment with Colombian payment preferences and regulations
- **Cost efficiency**: Competitive rates for local transactions, especially smaller amounts typical of event tickets
- **User experience**: Familiar payment methods increase conversion rates
- **Compliance**: PCI DSS compliance handled by Wompi, reducing our compliance burden
- **Development velocity**: Well-documented API enables rapid integration
- **Real-time updates**: Webhook notifications provide immediate payment confirmation
- **Mobile optimization**: Native mobile payment flows improve user experience
- **Multi-method support**: Comprehensive coverage of Colombian payment landscape

**Negative:**

- **Vendor lock-in**: Tight coupling with Wompi-specific API and features
- **Geographic limitation**: Difficult expansion to other countries without additional integrations
- **API limitations**: Rate limiting and feature constraints compared to global providers
- **Single point of failure**: Dependency on Wompi's service availability

**Risk mitigation strategies:**

- **Graceful degradation**: Handle Wompi service outages with appropriate user messaging
- **Monitoring**: Comprehensive monitoring of Wompi API response times and error rates
- **Backup communication**: Alternative channels for payment status in case of webhook failures
- **Testing**: Comprehensive testing with Wompi sandbox environment
- **Documentation**: Maintain detailed integration documentation for future modifications

**Financial impact:**

- **Processing fees**: 3.4% + $700 COP for card transactions, lower for PSE and digital wallets
- **Monthly costs**: No monthly fees, pay-per-transaction model
- **Development costs**: Estimated 2-3 weeks of development time vs. 3-6 months for custom solution
- **Compliance costs**: Avoided PCI DSS certification costs (~$50,000+ annually)

**Integration phases:**

- **Phase 1**: Basic card payment processing with webhook handling
- **Phase 2**: PSE and digital wallet integration (Nequi, Daviplata)
- **Phase 3**: Cash payment methods (Efecty, Baloto)
- **Phase 4**: Advanced features (tokenization, recurring payments)

**Security considerations:**

- **No card data storage**: All sensitive payment data handled by Wompi
- **Webhook validation**: Cryptographic signature verification for all webhooks
- **HTTPS enforcement**: All Wompi API communication over secure connections
- **PCI DSS scope**: Reduced PCI DSS scope due to external payment processing
- **Audit logging**: Complete audit trail for all payment operations

---

## References

- Payment module implementation → `modules/payment/`
- Wompi API documentation → https://docs.wompi.co/
- Payment security requirements → `evidence/security-threat-model.md#payment-information-disclosure`
- Order processing flow → `modules/ticket/application/usecase/ProcessOrderUseCase.java`
- Webhook handling → `modules/payment/infrastructure/external/WompiWebhookController.java`
- Payment audit logging → `evidence/cross-cutting.md#audit-logging`

# ADR-007 — WebSocket Real-time Communication

| Field | Value |
|---|---|
| **ID** | ADR-007 |
| **Date** | 2024-09-08 |
| **Status** | Accepted |
| **Authors** | Cristofer Lozano — Tech Lead, Jhampier Santos — DevOps |
| **Reviewers** | Development Team |

---

## Context

TuEvento requires real-time seat reservation updates to prevent conflicts when multiple users select seats simultaneously. The system needs:

- **Immediate feedback**: Seat status changes visible to all users within 100ms
- **Cross-browser compatibility**: Support for older browsers and network restrictions
- **Mobile support**: Reliable connections on mobile networks with intermittent connectivity
- **Scalability**: Handle multiple concurrent events with hundreds of users per event
- **Connection management**: Graceful handling of connection drops and reconnections
- **Security**: Authenticated WebSocket connections with proper authorization
- **Fallback support**: Degraded functionality when WebSocket unavailable

The core use case is seat selection where users must see real-time updates of seat availability to avoid booking conflicts.

---

## Evaluated alternatives

### Alternative A — WebSocket with SockJS + STOMP (CHOSEN)

#### Spring WebSocket with SockJS fallback and STOMP messaging protocol

- **Pros:**
  - Cross-browser compatibility with automatic fallback (WebSocket → SSE → polling)
  - Spring WebSocket provides seamless integration with Spring Security
  - STOMP protocol supports topic subscriptions (per-event channels)
  - SockJS handles connection recovery and network issues automatically
  - Message acknowledgment and reliability features
  - Built-in support for authentication and session management
  - Redis pub/sub integration for horizontal scaling

- **Cons:**
  - More complex than raw WebSockets
  - Additional protocol overhead with STOMP framing
  - SockJS fallbacks have higher latency than pure WebSocket

## Alternative B — Native WebSocket API

#### Raw WebSocket implementation without frameworks

- **Pros:**
  - Lowest latency and overhead
  - Full control over connection and message handling
  - No additional dependencies
- **Cons:**
  - No fallback for incompatible browsers or networks
  - Manual connection recovery and heartbeat implementation
  - Complex integration with Spring Security
  - Mobile network reliability issues
  - Custom protocol design and testing required

## Alternative C — Server-Sent Events (SSE)

#### One-way server-to-client communication with EventSource

- **Pros:**
  - Simpler than WebSocket, HTTP-based
  - Good browser support and automatic reconnection
  - Works through proxies and firewalls
- **Cons:**
  - One-way communication only (server → client)
  - Requires separate endpoint for client actions
  - Less efficient for bidirectional communication
  - Limited message types compared to WebSocket

## Alternative D — HTTP Polling

#### Regular AJAX requests to check for seat updates

- **Pros:**
  - Simple implementation with standard HTTP
  - No connection management complexity
  - Works in any network environment
- **Cons:**
  - High latency (polling interval vs real-time)

- Increased server load and bandwidth usage
- Poor user experience for real-time interactions
- Battery drain on mobile devices

### Alternative E — Long Polling

**HTTP requests held open until updates are available**

- **Pros:**
  - Lower latency than regular polling
  - HTTP-based, works through firewalls
  - Automatic fallback behavior
- **Cons:**
  - Connection timeout handling complexity
  - Higher server resource usage (held connections)
  - Less efficient than WebSocket for frequent updates
  - Proxy and load balancer complications

---

## Decision

**Alternative A:** WebSocket with SockJS + STOMP protocol

**Technical implementation:**

- **Primary**: WebSocket for modern browsers with good connectivity
- **Fallback 1**: Server-Sent Events for browsers without WebSocket support
- **Fallback 2**: XHR streaming for older browsers
- **Fallback 3**: XHR polling as last resort

**Message protocol:**

```
// Subscribe to event seat updates
CONNECT /ws/seats
SUBSCRIBE /topic/events/{eventId}/seats

// Seat status update message
{
  "type": "SEAT_STATUS_CHANGED",
  "eventId": "uuid",
  "seatId": "uuid",
  "status": "RESERVED|AVAILABLE|SOLD",
  "userId": "uuid",
  "timestamp": "2024-09-08T10:30:00Z",
  "expiresAt": "2024-09-08T10:45:00Z" // for reservations
}
```

```
// Heartbeat for connection monitoring
{
  "type": "HEARTBEAT",
  "timestamp": "2024-09-08T10:30:00Z"
}
```

**Security model:**

- **Authentication**: JWT token validation for WebSocket handshake
- **Authorization**: Users can only subscribe to events they have access to
- **Rate limiting**: Maximum message rate per connection to prevent abuse
- **Connection limits**: Maximum concurrent connections per user

**Scaling strategy:**

- **Redis pub/sub**: Distribute messages across multiple application instances
- **Event-specific topics**: Users subscribe only to relevant event updates
- **Connection management**: Track active connections per event in Redis
- **Load balancing**: Sticky sessions for WebSocket connections

---

## Consequences

**Positive:**

- **Real-time experience**: Sub-100ms seat status updates provide immediate feedback
- **Cross-browser support**: SockJS ensures functionality across all target browsers
- **Mobile reliability**: Automatic reconnection handles mobile network issues
- **Scalable architecture**: Redis pub/sub enables horizontal scaling
- **Security integration**: Seamless authentication with existing JWT system
- **Developer experience**: Spring WebSocket provides familiar programming model
- **Graceful degradation**: Fallback mechanisms ensure basic functionality always works

**Negative:**

- **Complexity**: More complex than simple HTTP polling solutions
- **Resource usage**: WebSocket connections consume server memory
- **Network requirements**: Some corporate firewalls may block WebSocket connections
- **Testing challenges**: Real-time features require specialized testing approaches

**Performance characteristics:**

- **Latency**: < 100ms for seat status updates under normal conditions
- **Throughput**: Support 500+ concurrent connections per application instance
- **Reliability**: 99.9% message delivery with SockJS fallbacks

- **Bandwidth**: ~50% reduction compared to polling equivalent

**Implementation phases:**

- **Phase 1**: Basic WebSocket connection with seat status broadcasts
- **Phase 2**: Authentication and per-event topic subscriptions
- **Phase 3**: Redis pub/sub for multi-instance deployment
- **Phase 4**: Advanced features (connection monitoring, rate limiting)

**Monitoring and diagnostics:**

- **Connection metrics**: Active connections, connection duration, failure rates
- **Message metrics**: Messages per second, delivery success rates
- **Performance monitoring**: Message latency, memory usage per connection
- **Error tracking**: Connection failures, authentication failures, message delivery issues

**Fallback behavior:**

- **WebSocket unavailable**: Automatic SockJS fallback to SSE or polling
- **Authentication failure**: Connection rejected with clear error message
- **Server overload**: Graceful connection limiting with user notification
- **Network issues**: Client-side reconnection with exponential backoff

---

## References

- Seat management real-time requirements → `04-requirements/evidence/functional.md#FR-024-026`
- WebSocket security configuration → `modules/seat/interfaces/websocket/`
- Redis pub/sub setup → `ADR-004-postgresql-redis-strategy.md`
- Authentication integration → `ADR-005-jwt-refresh-token-auth.md`
- Load testing strategy → `12-testing/performance-tests.md`